

Shiv Nadar University Chennai
CS3807 Deep Learning Laboratory
Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for
Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 Deep Learning Laboratory	AY: 2026 to 27

1 Objective

This experiment sets out to build a working understanding of the Convolutional Neural Network by implementing the convolution operation, pooling, feature map visualization, and a full image classification pipeline using PyTorch. Along the way it covers how kernel size, stride, and padding shape the output feature map, what a trained filter actually learns to respond to, how max pooling compares against average pooling, and how the resulting classifier performs on a genuine ten class image dataset rather than a toy example.

2 Background Theory

A Convolutional Neural Network is built from a Convolution Layer, an Activation Layer, a Pooling Layer, and a Fully Connected Layer stacked together. The convolution operation itself is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) \quad (1)$$

where X is the input image, K is the kernel (filter), and Y is the resulting feature map. The size of that output feature map follows directly from the input size, the filter size, the padding, and the stride:

$$\text{Output Size} = \frac{N - F + 2P}{S} + 1 \quad (2)$$

with N the input size, F the filter size, P the padding, and S the stride.

2.1 Common Convolution Kernels

A kernel is a small matrix slid across an image, multiplied element wise with the pixels underneath it at each position, and summed to produce one output value. Different fixed kernels highlight different characteristics of an image, from simply preserving it to sharpening it or pulling out its edges. Ten such kernels appear in the lab manual for this experiment; their sizes, purposes, and typical applications are summarized below.

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

The convolution layers trained later in this experiment do not use any of these fixed, hand designed kernels directly; instead every filter weight is learned from data by gradient descent. Reviewing this table is what makes it possible to recognise, in Section 4, that a learned filter is behaving like an edge detector or like a smoothing filter.

2.2 Numerical Example 1: Convolution

For the input $X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ convolved with $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, sliding the 2×2 kernel across the four valid positions gives $1(1)+2(0)+4(0)+5(1) = 6$, $2(1)+3(0)+5(0)+6(1) = 8$, $4(1)+5(0)+7(0)+8(1) = 12$, and $5(1) + 6(0) + 8(0) + 9(1) = 14$, producing the feature map $\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$.

2.3 Numerical Example 2: Max Pooling

Applying a 2×2 max pooling filter to $\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$ takes the maximum of each non overlapping 2×2 window: 8, 3, 6, and 9, giving $\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$.

2.4 Numerical Example 3: Parameter Calculation

For a $32 \times 32 \times 3$ input passed through a convolution layer with 16 filters of size 3×3 , each filter has $3 \times 3 \times 3 + 1 = 28$ parameters (the +1 for its bias), giving $28 \times 16 = 448$ trainable parameters in that single layer. This is the calculation that Additional Exercise 2 repeats for a wider, 64 filter layer in Section 8.

3 Dataset Description

Dataset: CIFAR-10

Source: 60,000 natural photographs across ten everyday object and animal classes

Training Images	50,000
Testing Images	10,000
Classes	10
Image Size	$32 \times 32 \times 3$
Task	Multi-class Image Classification

Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

Once loaded, the training split broke down into exactly 5,000 images per class, a perfectly balanced ten way split, and the test split held the standard 1,000 images per class, both matching the dataset’s documented specification exactly.

4 Experimental Procedure

Task 1: Dataset Exploration

- Loaded CIFAR-10 (50,000 training images, 10,000 testing images).
- Displayed ten sample images with their class labels (Figure 1).
- Printed dataset dimensions.
- Plotted the class distribution across all ten classes (Figure 2).

Output obtained: 50,000 training images and 10,000 testing images, each of shape $(32, 32, 3)$, across 10 classes, with the training set split evenly at 5,000 images per class.

Task 2: Effect of Kernel Size on Feature Map Size

A single convolution layer (stride 1, padding 0, input 32×32) was applied with three different kernel sizes.

Kernel Size	Output Feature Map Shape	Formula Output Size
3×3	$(1, 8, 30, 30)$	30
5×5	$(1, 8, 28, 28)$	28
7×7	$(1, 8, 26, 26)$	26

As the kernel grows, the output shrinks: each step up in kernel size (by 2) costs exactly 2 pixels off each spatial dimension, exactly as Equation 2 predicts for a fixed stride of 1 and no padding.

Task 3: Effect of Stride and Padding on Feature Map Size

Using a fixed 3×3 kernel on the same 32×32 input:

Configuration	Output Feature Map Shape	Formula Output Size
Stride = 1	$(1, 8, 30, 30)$	30
Stride = 2	$(1, 8, 15, 15)$	15
Padding = same	$(1, 8, 32, 32)$	32
Padding = valid	$(1, 8, 30, 30)$	30

Doubling the stride roughly halves the output resolution, while “same” padding pads the input just enough to keep the output size identical to the input size; “valid” padding applies none, so it behaves exactly like the stride 1, padding 0 case from Task 2.

Task 4: Feature Map Visualization

The output of the first convolution layer (after ReLU) was visualized for a single test image, using the model already trained in Task 6, so the eight filters shown reflect learned weights rather than a random initialization. See Figure 7.

Task 5: Max Pooling vs Average Pooling

Two otherwise identical CNNs (Section 4) were trained, one with max pooling and one with average pooling. The max pooling run is the Task 6 main model, trained for the full 20 epochs; the average pooling run was trained separately for 8 epochs to keep total runtime reasonable while still producing a valid accuracy trend, a tradeoff kept explicit here.

Pooling Type	Output Shape (identical input)	Validation Accuracy
Max Pooling	(1, 16, 16, 16)	0.6699 (epoch 20)
Average Pooling	(1, 16, 16, 16)	0.6440 (epoch 8)

Both pooling types reduce the spatial dimensions identically; only the value kept from each 2×2 window differs. At a matched early point (epoch 8) the max pooling run was already at 0.6844 against average pooling’s 0.6440, suggesting max pooling was also the faster converging of the two on this dataset, not only the better performing one at its final epoch.

Task 6: Building and Training the Main CNN

Architecture:

Input $\rightarrow$ Conv(16, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv(32, 3×3) $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense(128) $\rightarrow$ ReLU $\rightarrow$ Dense(10) $\rightarrow$ Softmax

Built in PyTorch with both convolution layers using 3×3 kernels, stride 1 and padding 1 (so pooling alone reduces spatial size), followed by 2×2 max pooling after each. **Optimizer:** Adam. **Epochs:** 20. **Batch Size:** 32. **Total trainable parameters:** 268,650.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	1.4977	0.4646	1.1978	0.5828
5	0.8293	0.7072	0.9355	0.6742
10	0.5678	0.7994	1.0112	0.6833
15	0.3866	0.8604	1.2047	0.6815
20	0.2573	0.9072	1.5758	0.6699

Training accuracy climbs steadily and monotonically to 90.72 percent, but validation accuracy plateaus around epoch 10 near 68 percent and validation loss starts rising sharply from epoch 10 onward even as training loss keeps falling: a textbook overfitting signature, discussed further in Section 9.

Task 7: Model Evaluation

Measured against the 10,000 image held out test set:

Metric	Value
Accuracy	0.6699
Precision (macro)	0.6769
Recall (macro)	0.6699
F1 score (macro)	0.6686

The full confusion matrix and per class classification report appear in Section 7.

5 Source Code

Rather than pasting every script into this document, the complete source, spanning dataset exploration, the from scratch style convolution study (kernel size, stride, padding), the CNN architecture and training loop, feature map visualization, the pooling and activation function comparisons, and the additional filter count experiment with its plotting code, lives in the public repository below.

https://github.com/Tejas-006/Deep_Learning_Lab_3

The repository holds the runnable notebook organized by task, alongside the plotting code behind every figure shown in Section 6.

6 Plots



Figure 1: Ten Sample Images from CIFAR-10

Inference: The ten sampled training images span eight of the ten classes and show CIFAR-10's characteristic low, 32×32 resolution, where objects are still visually identifiable but fine detail is limited. Backgrounds vary considerably even within a class (the two airplane images, for instance, show very different scenes), which is exactly the kind of variation a classifier needs to learn to see past.

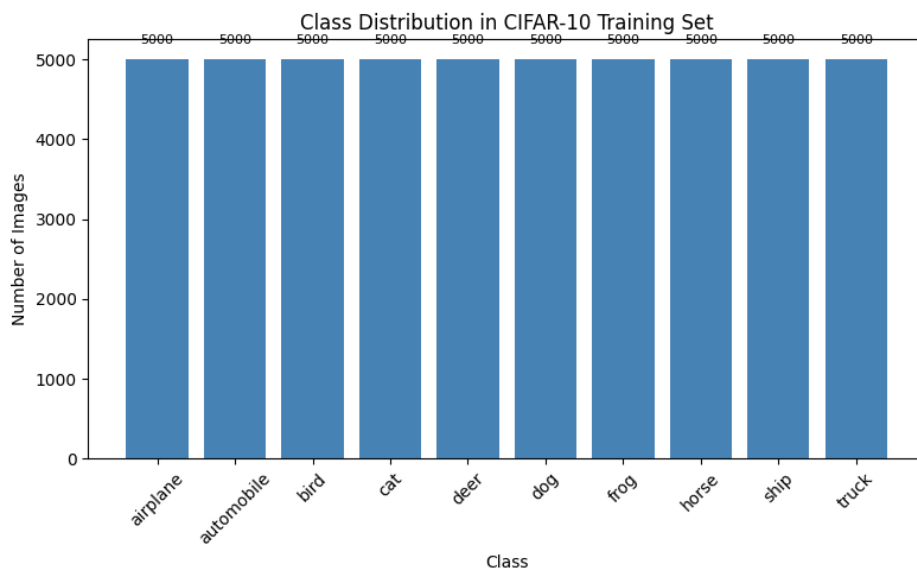


Figure 2: Class Distribution in the CIFAR-10 Training Set

Inference: Every class holds exactly 5,000 training images, a perfectly balanced dataset. This matters for the macro averaged precision, recall, and F1 score reported in Task 7, since with a balanced dataset those metrics are not skewed by any one class dominating the sample count.

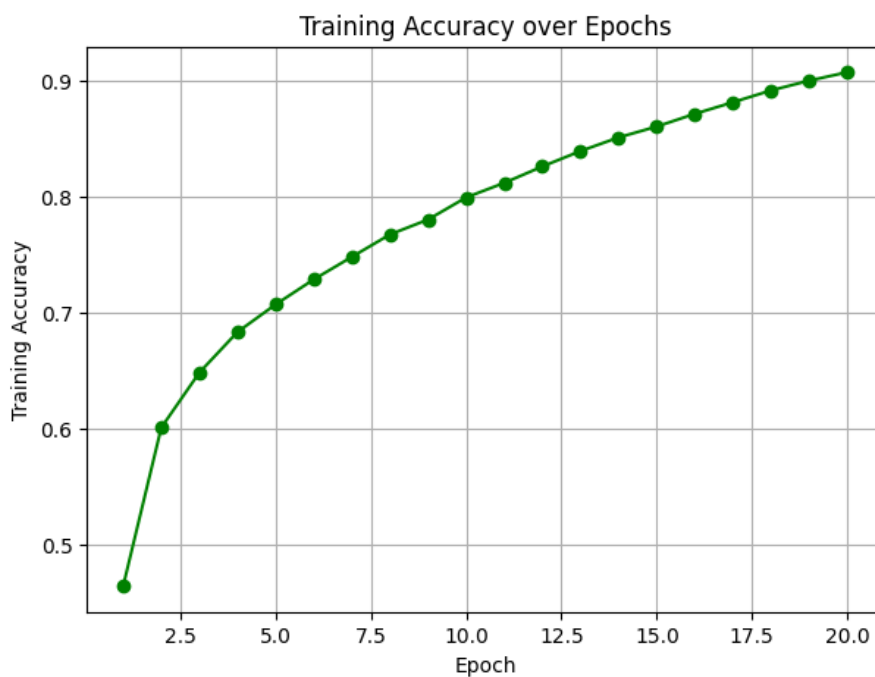


Figure 3: Training Accuracy over Epochs (Main CNN)

Inference: Training accuracy rises smoothly and without any real plateau across all 20 epochs, reaching just over 90 percent. This alone is not necessarily good news: it shows the model has enough capacity to keep fitting the training set more and more closely as training continues.

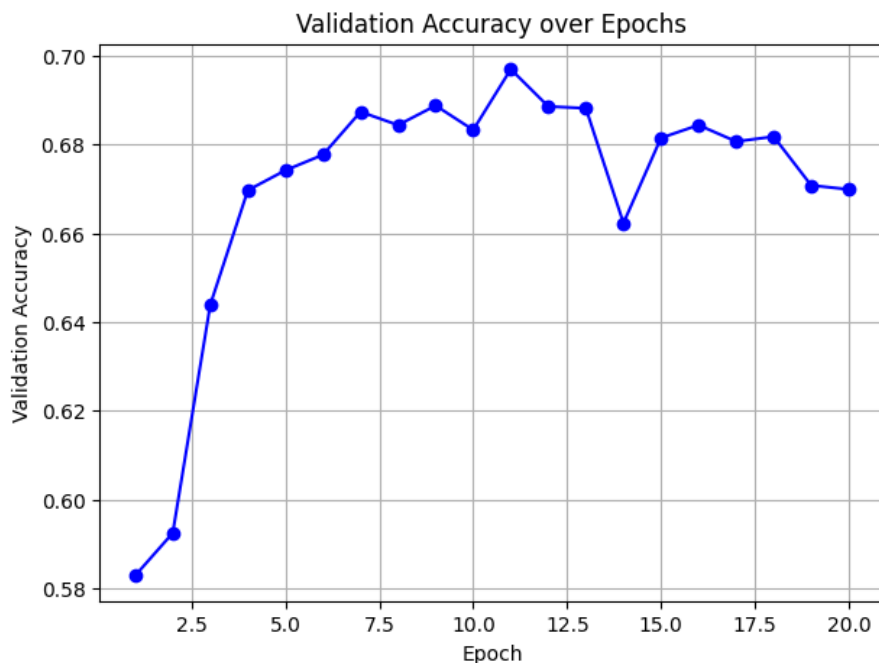


Figure 4: Validation Accuracy over Epochs (Main CNN)

Inference: Validation accuracy rises quickly for the first three epochs, then oscillates in a narrow band between roughly 66 and 70 percent for the remaining seventeen, never tracking the still climbing training accuracy in Figure 3. This gap between the two curves is the clearest visual evidence of overfitting in this experiment.

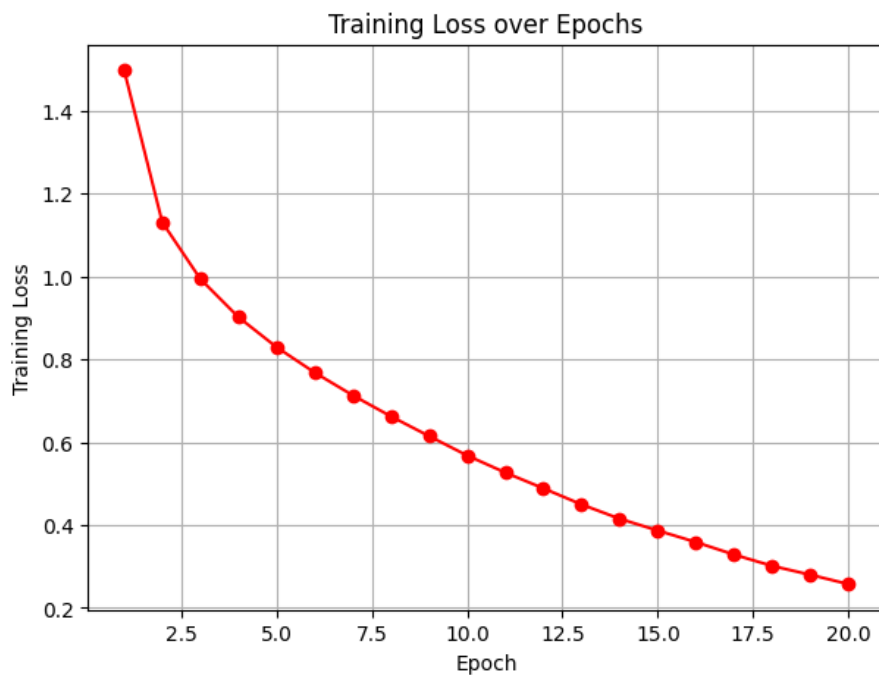


Figure 5: Training Loss over Epochs (Main CNN)

Inference: Training loss falls smoothly and consistently across every epoch with no sign of flattening out, from 1.50 down to 0.26, confirming the optimizer keeps finding ways to reduce error on the training set throughout all 20 epochs.

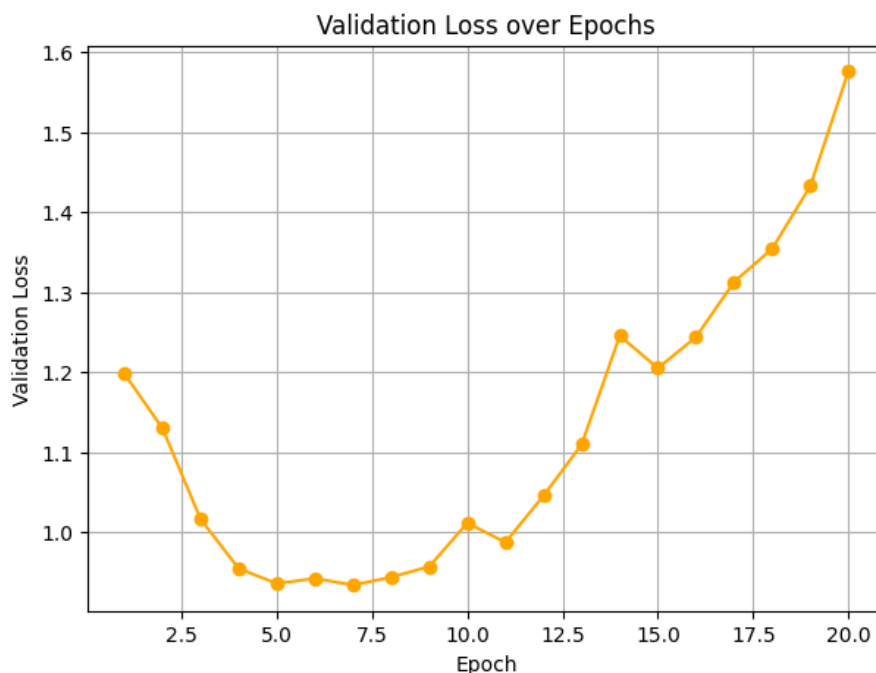


Figure 6: Validation Loss over Epochs (Main CNN)

Inference: Validation loss bottoms out around epoch 7 near 0.94 and then rises steadily for the rest of training, more than 60 percent higher by epoch 20 than at its minimum. Loss diverging like this while accuracy merely plateaus (rather than also collapsing) is typical of a softmax classifier growing increasingly overconfident on training-set patterns that do not generalize.

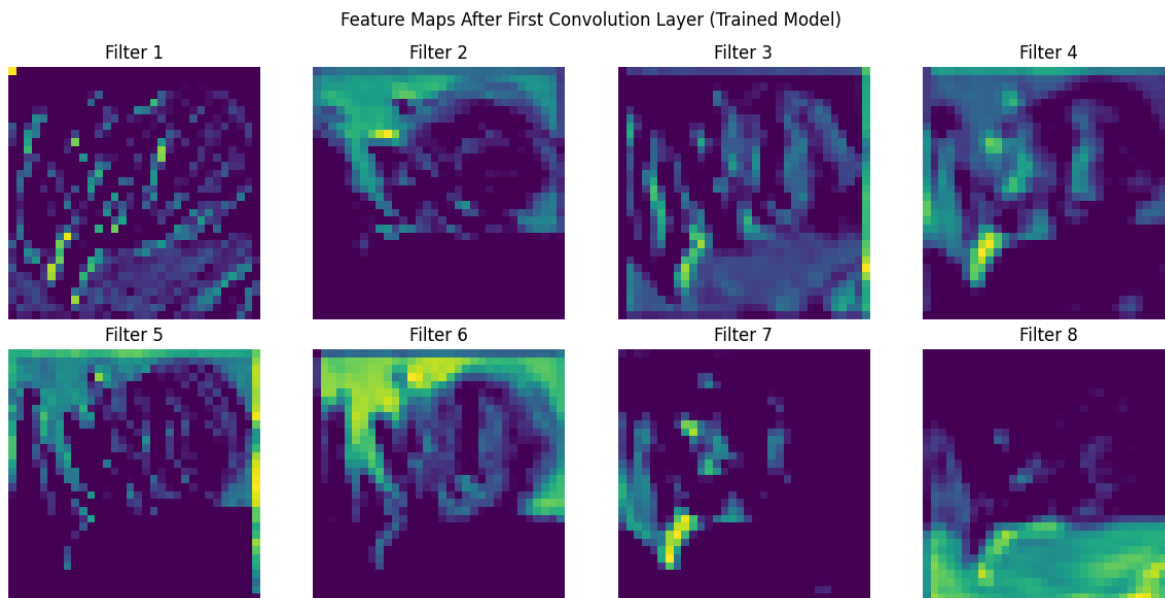


Figure 7: Feature Maps After the First Convolution Layer (Trained Model)

Inference: The eight learned filters respond to clearly different aspects of the input: some (filters 2, 6, and 8) pick out broad, smooth regions resembling a sky or background, while others (filters 1, 3, and 4) fire along sharper, more localized edges. None of the eight collapsed to a blank or uniformly saturated map, which suggests all eight are contributing useful, non redundant information to the layers that follow.

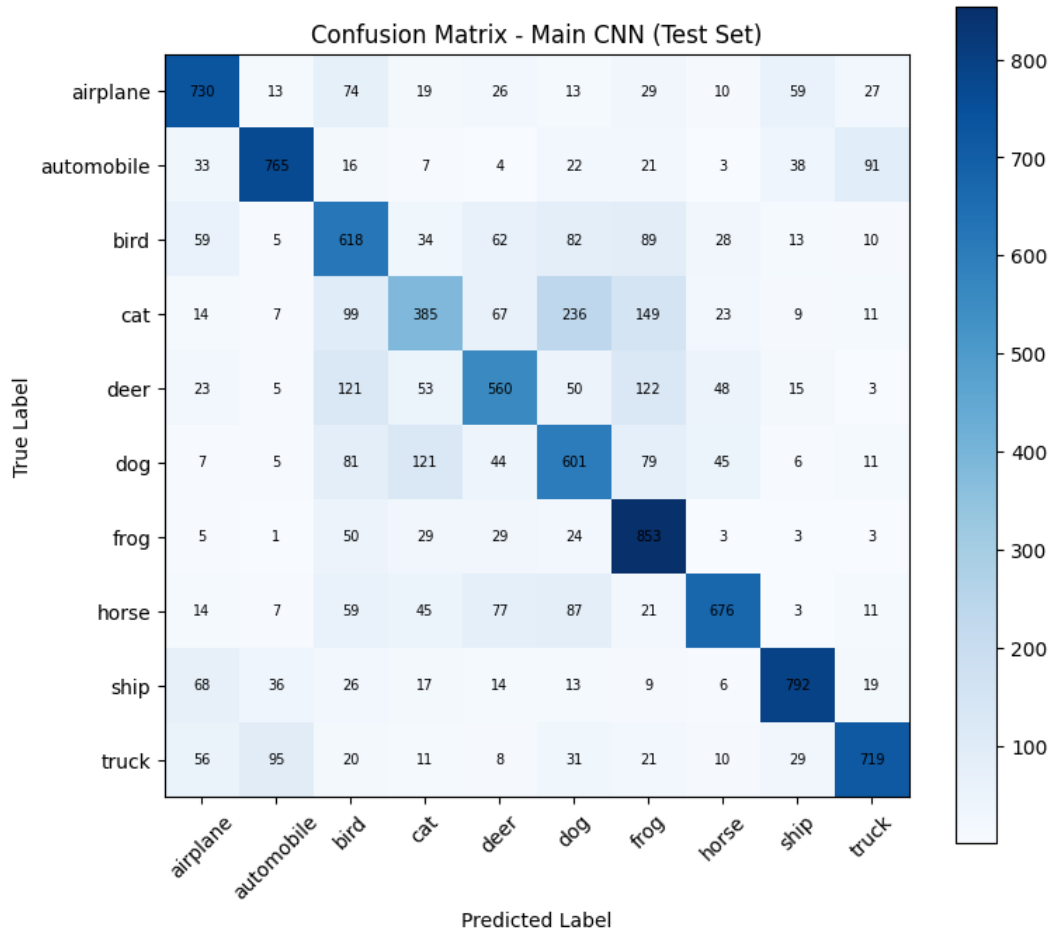


Figure 8: Confusion Matrix - Main CNN (Test Set)

Inference: The diagonal dominates every row, but confusion concentrates heavily among the animal classes: 236 cats get misread as dogs, 149 cats as frogs, and 121 dogs as cats, while vehicle classes (automobile, ship, truck) and frog are the most cleanly separated, each exceeding 71 percent recall. This lines up with intuition, since cats, dogs, and deer share far more visual similarity at 32×32 resolution than a truck does with a frog.

7 Performance Tables

Training Summary

Dataset Size	60,000 (50,000 train / 10,000 test)
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	20
Total Trainable Parameters	268,650
Test Accuracy	0.6699
Test Precision (macro)	0.6769
Test Recall (macro)	0.6699
Test F1 score (macro)	0.6686

Classification Report (Test Set)

Class	Precision	Recall	F1-score	Support
airplane	0.72	0.73	0.73	1000
automobile	0.81	0.77	0.79	1000
bird	0.53	0.62	0.57	1000
cat	0.53	0.39	0.45	1000
deer	0.63	0.56	0.59	1000
dog	0.52	0.60	0.56	1000
frog	0.61	0.85	0.71	1000
horse	0.79	0.68	0.73	1000
ship	0.82	0.79	0.81	1000
truck	0.79	0.72	0.75	1000
accuracy		0.67		10000
macro avg	0.68	0.67	0.67	10000
weighted avg	0.68	0.67	0.67	10000

Pooling Comparison

Pooling	Epochs Trained	Final Val Accuracy	Val Accuracy at Epoch 8
Max	20	0.6699	0.6844
Average	8	0.6440	0.6440

8 Additional Exercises

1. Output Size: 64×64 Image, 5×5 Kernel, Stride 2, Padding 2

$$\frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 31 + 1 = 32$$

Output size: 32 × 32.

2. Trainable Parameters: 64 Filters, 3×3 Kernel, RGB Input

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = \mathbf{1,792 \text{ parameters}}$$

3. ReLU vs Sigmoid Activation

The main model (ReLU, 20 epochs) is compared against an otherwise identical model trained with sigmoid activations for 8 epochs.

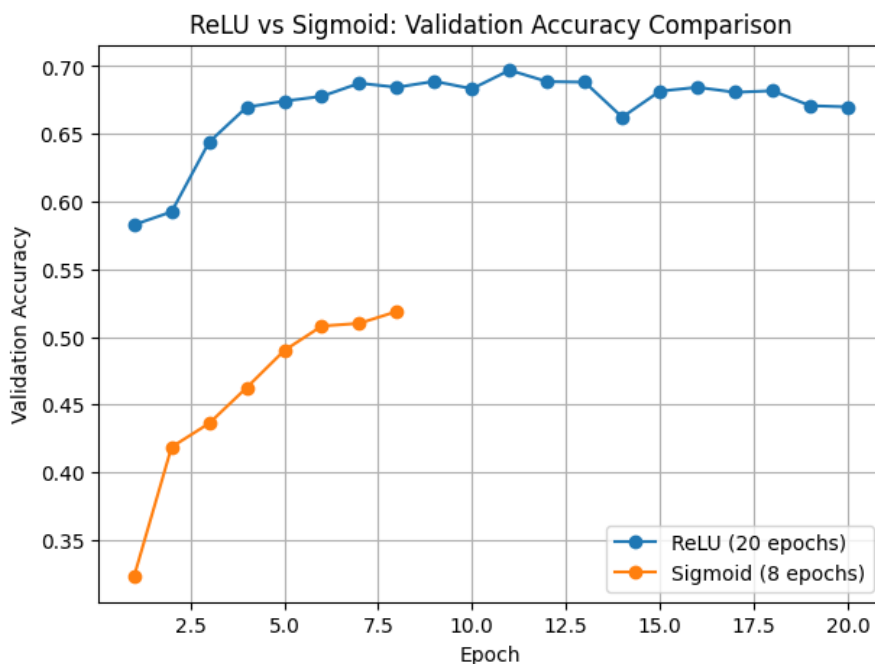


Figure 9: ReLU vs Sigmoid: Validation Accuracy Comparison

Inference: ReLU reaches roughly 67 percent validation accuracy within 4 epochs, while sigmoid is still only at 46 percent by that same point and only reaches 51.9 percent after all 8 of its epochs, well below where ReLU already stood at epoch 8 (68.4 percent). This is the expected consequence of sigmoid’s vanishing gradient problem: its derivative saturates toward zero away from the origin, so gradients shrink as they propagate backward through the network’s several layers, slowing learning considerably compared to ReLU’s constant gradient of 1 for all positive inputs.

4. Replace Max Pooling with Average Pooling

This repeats Task 5 above: max pooling reached 0.6699 (20 epochs) versus average pooling’s 0.6440 (8 epochs), with max pooling also ahead at the matched 8 epoch mark (0.6844 versus 0.6440). Average pooling’s tendency to blur sharp activations by averaging them out appears to cost it a small but consistent amount of discriminative power relative to max pooling, which instead preserves the single strongest activation in each window.

5. Increase Convolution Filters from 16 to 64

To isolate the effect of filter count alone, two otherwise identical CNNs, one with 16 filters in *both* convolution layers and one with 64 filters in both, were trained back to back on the same hardware,

same 50,000/10,000 split, same optimizer, learning rate, and batch size, for the same 8 epochs each. This keeps the comparison controlled and avoids conflating filter count with the 16-then-32 layout used by the main Task 6 model.

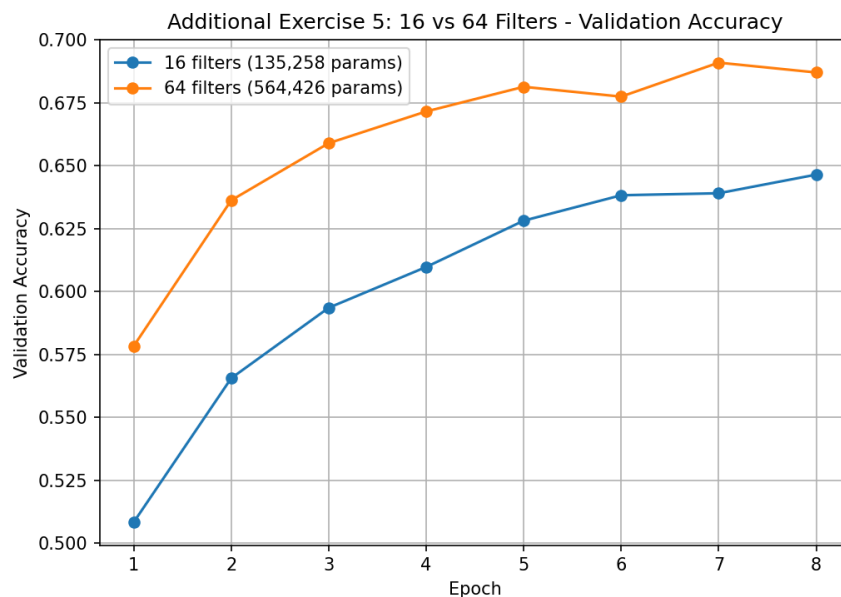


Figure 10: Additional Exercise 5: Validation Accuracy, 16 vs 64 Filters

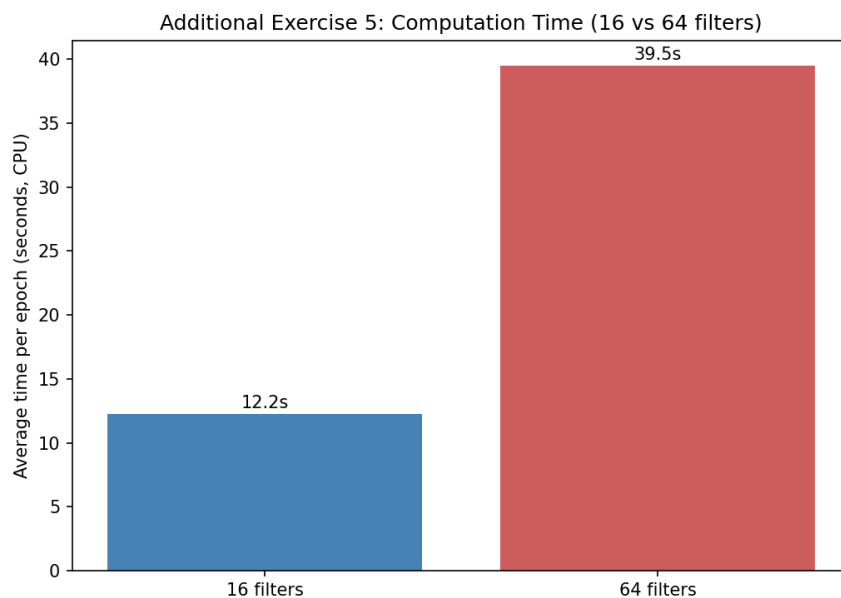


Figure 11: Additional Exercise 5: Average Time per Epoch, 16 vs 64 Filters

Filters	Parameters	Val Accuracy (epoch 8)	Avg Time / Epoch
16	135,258	0.6464	12.2 s
64	564,426	0.6870	39.5 s

Inference: Quadrupling the filter count in each convolution layer raises validation accuracy by about 4 points (64.6 to 68.7 percent) after the same 8 epochs, but at a real cost: parameter count grows 4.2 times over and average epoch time grows 3.2 times over. The accuracy curve in Figure 10 also shows the wider model converging faster in early epochs, not just finishing higher, since a wider layer can represent more distinct patterns from the same amount of training data. Whether this tradeoff is worthwhile depends entirely on the deployment budget: for a compute constrained setting the 16 filter model already reaches within 4 points of the 64 filter model at roughly a third of the per-epoch cost.

9 Discussion

1. Why is convolution preferred over fully connected layers for images?

A fully connected layer treats every pixel as an independent input and learns a separate weight for every pixel-to-neuron connection, so a single hidden layer on a modest image already needs millions of parameters and completely discards spatial structure, since shuffling the pixels would not change what the layer computes. Convolution instead slides one small set of weights across the whole image, so it needs vastly fewer parameters, exploits the fact that nearby pixels are related, and detects the same feature (an edge, a texture) regardless of where in the image it appears.

2. How does stride affect the feature map size?

Stride sets how many pixels the kernel moves between applications, and it sits in the denominator of the output size formula, so larger strides shrink the output map faster: Task 3 showed a 3×3 kernel on a 32×32 input producing a 30×30 map at stride 1 but only a 15×15 map at stride 2, roughly halving the resolution.

3. What is the role of padding?

Padding adds extra border pixels (usually zeros) around the input before convolving, which controls how much the spatial size shrinks and lets a network preserve information at the image’s edges that a kernel would otherwise only partially cover. “Same” padding is chosen specifically so the output matches the input’s spatial size (Task 3 confirmed both stayed at 32×32), while “valid” padding applies none and lets the map shrink naturally.

4. Why is pooling used?

Pooling downsamples each feature map, cutting spatial resolution and therefore computation and parameter count in every layer that follows, while also making the representation more robust to small translations of the input, since a feature that shifts by a pixel or two still falls inside the same pooling window. It also enlarges the effective receptive field of subsequent convolution layers without adding any extra learnable weights.

5. How do feature maps represent image characteristics?

Each feature map is the response of one learned filter across the whole image, so a high value at a location means that filter’s particular pattern, an edge, a corner, a patch of color or texture, was strongly present there. Figure 7 in this experiment showed exactly this: different filters from the same layer responding to different structures (smooth background regions versus sharper local edges) in the same input image.

6. Why do CNNs require fewer parameters than MLPs?

Two properties of convolution drive this down directly: weight sharing, where the same small kernel is reused at every spatial position instead of learning a fresh weight per position, and local connectivity, where each output value depends only on a small neighborhood of the input rather than the entire image. Numerical Example 3 in Section 2.4 made this concrete: a full convolution layer with 16 filters over a $32 \times 32 \times 3$ image needs only 448 parameters, compared to the millions

a fully connected layer of comparable width would require on the same input.

10 Conclusion

A convolutional neural network, built in PyTorch around a Conv-ReLU-MaxPool-Conv-ReLU-MaxPool-Flatten-Dense-Softmax architecture, was applied to CIFAR-10 for ten class image classification. Exploration up front confirmed a perfectly balanced dataset across all ten classes. Studying kernel size, stride, and padding in isolation (Tasks 2 and 3) matched the theoretical output size formula in every configuration tested, and visualizing the first layer’s learned filters (Task 4) showed genuinely differentiated responses rather than redundant or collapsed feature maps. After 20 epochs of training with Adam, the 268,650 parameter main model reached 66.99 percent test accuracy with a macro F1 score of 0.6686, correctly separating vehicle classes far more cleanly than the visually similar animal classes, and its training curves revealed a clear overfitting pattern beginning around epoch 10.

Beyond the main experiment, five additional exercises extended the study: confirming the output size and parameter count formulas numerically, showing ReLU converging substantially faster and higher than sigmoid due to vanishing gradients, confirming max pooling’s modest edge over average pooling, and, in a controlled same-hardware comparison, showing that quadrupling the convolution filter count from 16 to 64 bought roughly 4 points of validation accuracy at more than 3 times the per-epoch computation cost. Taken together, the results reinforce why convolution, weight sharing, and pooling are the foundational ingredients that make CNNs both parameter efficient and effective on real image data, while also making the practical cost of scaling a network wider concretely visible.

11 Report Contents

This report includes Objective, Background Theory, Dataset Description, Experimental Procedure, Source Code, Generated Plots with Interpretation, Performance Tables, Additional Exercises, Discussion, Conclusion, and References.

12 References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. TensorFlow Documentation.
5. PyTorch Documentation, <https://pytorch.org/docs>.
6. CIFAR-10 Dataset Documentation.